TP — Forensic Mémoire

Master CDSI — M1 — Investigation numérique

Université — Année universitaire 2025-2026

Sommaire

Avant-propos

1. Cadre général

1.1 Environnement imposé

1.2 Wrapper de lancement

1.3 Hygiène d'investigation

1.4 Format de rendu

2. Cheatsheet — Volatility 2 et Volatility 3

2.1 Identification du dump

2.2 Processus

2.3 Réseau

2.4 Système de fichiers et registre

2.5 Activité utilisateur

2.6 Détection de codes malveillants

2.7 Astuces génériques

2.8 Règles YARA — usage en forensic mémoire

3. Exercice 1 — « Never Too Late, Mister »

3.1 Contexte

3.2 Travail demandé

4. Exercice 2 — « Profil d'un poste »

4.1 Contexte

4.2 Travail demandé

5. Exercice 3 — « Obsession »

5.1 Contexte

5.2 Travail demandé

6. Exercice 4 — « Vendredi noir »

6.1 Contexte

6.2 Travail demandé

7. Exercice 5 — « Anatomie d'un ver SMB » *(approfondissement)*

7.1 Contexte

7.2 Travail demandé

8. Annexes

8.1 Calculer un SHA-256

8.2 Journaliser une session shell

8.3 Outils complémentaires autorisés

Avant-propos

Ce polycopié rassemble **quatre exercices indépendants** d'investigation sur dump mémoire. Chaque exercice repose sur une image RAM brute (`.raw`) qui vous est fournie séparément. Vous traiterez les exercices dans l'ordre indiqué par votre enseignant.

Aucun des exercices n'est résoluble par simple recherche en ligne du nom du fichier : un examen attentif des artefacts en mémoire est requis.

1. Cadre général

1.1 Environnement imposé

L'analyse passe par le conteneur **`cincan/volatility`**. Vous êtes libres de choisir entre **Volatility 2** (image `cincan/volatility`) et **Volatility 3** (image `cincan/volatility3`) ; l'important est de justifier votre choix dans le compte-rendu et de rester cohérent au sein d'un même exercice.

```
docker pull cincan/volatility:latest    # Volatility 2.6
docker pull cincan/volatility3:latest   # Volatility 3 (Vol3)
```

1.2 Wrapper de lancement

Le script `vol.sh` qui vous est fourni encapsule l'appel Docker en montant le répertoire courant sur `/data` à l'intérieur du conteneur, en lecture seule, ce qui garantit qu'aucune commande ne peut altérer le dump.

```
chmod +x vol.sh
./vol.sh -f /data/<dump>.raw imageinfo            # Vol2
VOL_IMAGE=cincan/volatility3 ./vol.sh -f /data/<dump>.raw windows.info  # Vol3
```

1.3 Hygiène d'investigation

- Le fichier `.raw` ne doit **jamais** être modifié. Calculez et consignez son SHA-256 avant et après chaque exercice.
- Toute extraction (fichier, processus, dump de processus, hash) sera rangée dans un sous-dossier `extractions/` et accompagnée de son SHA-256.
- Aucune connexion réseau sortante n'est autorisée vers les IPs identifiées dans les dumps.
- Les commandes exécutées seront systématiquement consignées (`script`, `tee`, ou journal manuel) en annexe du compte-rendu.

1.4 Format de rendu

Un compte-rendu PDF unique par exercice, contenant pour chaque question : la commande exécutée, l'extrait pertinent de la sortie, votre interprétation, la réponse demandée. Les flags trouvés seront recopiés sans capture d'écran.

2. Cheatsheet — Volatility 2 et Volatility 3

Cette annexe technique ne donne aucune indication sur les exercices : elle se contente de rappeler la correspondance entre les plugins des deux générations de l'outil.

2.1 Identification du dump

Objectif	Volatility 2	Volatility 3
Identifier le profil / l'OS	`imageinfo`	`windows.info`
Détails du KDBG (alternative)	`kdbgscan`	(intégré à `windows.info`)

> **Volatility 3** détecte automatiquement le profil grâce aux symboles ; il n'est plus nécessaire de passer `--profile=`.

2.2 Processus

Objectif	Volatility 2	Volatility 3
Liste linéaire des processus	`pslist`	`windows.pslist`
Arborescence parent/enfant	`pstree`	`windows.pstree`
Détection de processus masqués (cross-view)	`psscan`	`windows.psscan`
Lignes de commande complètes	`cmdline`	`windows.cmdline`
Variables d'environnement	`envars`	`windows.envars`
Handles ouverts par un processus	`handles -p <PID>`	`windows.handles --pid <PID>`
DLL chargées	`dlllist -p <PID>`	`windows.dlllist --pid <PID>`
Exporter (dumper) un processus	`procdump -p <PID> --dump-dir=.`	`windows.dumpfiles --pid <PID>`
Exporter la mémoire d'un processus	`memdump -p <PID> --dump-dir=.`	`windows.memmap --pid <PID> --dump`

2.3 Réseau

Objectif	Volatility 2	Volatility 3
Connexions actives (XP/2003)	`connections`, `connscan`	`windows.netscan`
Sockets ouverts (XP/2003)	`sockets`, `sockscan`	`windows.netscan`
État réseau (Vista et postérieur)	`netscan`	`windows.netscan` / `windows.netstat`

2.4 Système de fichiers et registre

Objectif	Volatility 2	Volatility 3
Énumération des fichiers résidant en mémoire	`filescan`	`windows.filescan`
Extraction d'un fichier identifié	`dumpfiles -Q <offset> --dump-dir=.`	`windows.dumpfiles --virtaddr <offset>`
Lister les ruches du registre	`hivelist`	`windows.registry.hivelist`
Imprimer une clé de registre	`printkey -K "Software\..."`	`windows.registry.printkey --key "Software\..."`
Programmes au démarrage	`printkey -K "Software\Microsoft\Windows\CurrentVersion\Run"`	`windows.registry.userassist`
Scans des entrées Userassist	`userassist`	`windows.registry.userassist`
Comptes locaux et hashes NTLM	`hashdump`	`windows.hashdump`
Secrets LSA	`lsadump`	`windows.lsadump`
Mots de passe en cache	`cachedump`	`windows.cachedump`

2.5 Activité utilisateur

Objectif	Volatility 2	Volatility 3
Captures d'écran à partir de la GUI	`screenshot --dump-dir=.`	(non disponible nativement)
Historique des consoles	`cmdscan`, `consoles`	`windows.cmdline` (partiel)
Presse-papiers	`clipboard`	(non disponible nativement)
Notepad ouverts (texte en mémoire)	`notepad` (Win7/x86)	(passer par `memmap` + `strings`)
Fenêtres GUI actives	`windows`, `wintree`	(non disponible nativement)

2.6 Détection de codes malveillants

Objectif	Volatility 2	Volatility 3
Pages exécutables suspectes / injection	`malfind`	`windows.malfind`
Hooks d'API et de SSDT	`apihooks`, `ssdt`	(non porté à ce jour)
Recherche YARA en mémoire	`yarascan -Y "..."`	`yarascan.yarascan -Y "..."`
Drivers chargés	`modules`, `modscan`	`windows.modules`
Services Windows	`svcscan`	`windows.svcscan`
Tâches planifiées (via registre)	`printkey` sur `\Schedule\TaskCache`	`windows.registry.printkey`

2.7 Astuces génériques

```
# Vol2 — appliquer un profil détecté
./vol.sh -f /data/dump.raw --profile=Win7SP1x64 pslist

# Vol2 — dumper un processus particulier
./vol.sh -f /data/dump.raw --profile=Win7SP1x64 \
    memdump -p 1234 --dump-dir=/data/extractions

# Vol3 — invocation type
VOL_IMAGE=cincan/volatility3 ./vol.sh -f /data/dump.raw windows.pslist

# Strings ciblé sur un dump de processus
strings -e l 1234.dmp | grep -i "flag\|http\|password"

# Extraction d'un fichier (Vol2)
./vol.sh -f /data/dump.raw --profile=Win7SP1x64 \
    dumpfiles -Q 0x000000003fc6c8f0 --dump-dir=/data/extractions
```

2.8 Règles YARA — usage en forensic mémoire

Principe

YARA est un moteur de **règles de détection par motifs** très utilisé en réponse à incident. Une règle YARA décrit un fichier ou une empreinte mémoire à l'aide d'une combinaison logique de chaînes (littérales, hexadécimales ou expressions régulières) et d'une condition. Volatility intègre YARA via le plugin `yarascan` (Vol2) et `yarascan.yarascan` (Vol3), ce qui permet de **balayer toute la mémoire physique ou la mémoire d'un processus précis** à la recherche d'indicateurs.

Anatomie d'une règle

```
rule Demonstration_Flag
{
    meta:
        author      = "etudiant.cdsi"
        description = "Repère un flag CTF en mémoire"
        date        = "2026-04-26"

    strings:
        $txt_flag    = "flag{" ascii nocase
        $txt_inctf   = "inctf{" ascii nocase
        $hex_pe      = { 4D 5A }                    // En-tête PE
        $re_b64flag  = /ZmxhZ3[A-Za-z0-9+\/]{8,}/   // "flag{" en base64

    condition:
        any of them
}
```

- Chaque chaîne est précédée d'un `$` et peut être **textuelle**, **hexadécimale** ou **régulière**.
- Le bloc `condition` combine ces chaînes avec des opérateurs logiques (`and` , `or` , `any of them` , `2 of ($txt_*)` , `at <offset>`).
- Le bloc `meta` est purement informatif mais **doit** être renseigné pour la traçabilité (auteur, date, source).

Pourquoi en forensic mémoire ?

1. **Recherche transverse** : la même règle peut être appliquée à un fichier sur disque, à une image mémoire, ou à un processus dumpé.
2. **Indicateurs de compromission** (IoC) : une règle YARA est fréquemment publiée avec un avis de sécurité ; la confronter à votre dump permet d'identifier rapidement la famille de malware en cause.
3. **Détection malgré les couches** : un binaire chiffré sur disque est nécessairement déchiffré en mémoire pour s'exécuter. YARA appliqué à la mémoire trouvera ce que YARA appliqué au disque ne trouvera pas.

Mise en pratique avec `yarascan`

```
# Vol2 — règle inline (recherche d'une chaîne)
./vol.sh -f /data/dump.raw --profile=Win7SP1x64 \
    yarascan -Y "flag{"

# Vol2 — règle inline (recherche binaire)
./vol.sh -f /data/dump.raw --profile=Win7SP1x64 \
    yarascan -Y "{ 50 4B 03 04 }"     # En-tête ZIP

# Vol2 — règle dans un fichier, ciblée sur un processus
./vol.sh -f /data/dump.raw --profile=Win7SP1x64 \
    yarascan -y /data/regles/iocs.yar -p 1984

# Vol3 — équivalent
VOL_IMAGE=cincan/volatility3 ./vol.sh -f /data/dump.raw \
    yarascan.yarascan --yara-rules /data/regles/iocs.yar
```

Bonnes pratiques

- Écrire au moins **deux** chaînes par règle pour limiter les faux positifs ; n'autoriser une règle à une seule chaîne que si celle-ci est très spécifique (chaîne longue et propre à la menace).
- Privilégier les chaînes **propres au comportement** (URL de C2, mutex, clé de registre) plutôt qu'aux chaînes volatiles (chemins utilisateurs, dates de compilation).
- **Documenter** la règle : l'absence de `meta` est éliminatoire pour un rapport recevable en justice.
- Toujours tester la règle sur un échantillon **bénin** afin de vérifier qu'elle ne provoque pas de faux positif.

À vous de jouer

Vous êtes invités, lorsque c'est pertinent, à **rédiger vos propres règles YARA** pour soutenir votre démarche d'investigation et à les joindre en annexe de votre compte-rendu.

3. Exercice 1 — « Never Too Late, Mister »

Dump associé : `exercice1.raw`

3.1 Contexte

Une équipe d'intervention a saisi le poste d'un utilisateur identifié par le pseudonyme `SmartNet` à la suite d'un signalement interne. Pendant la procédure de saisie, l'opérateur a constaté que la machine était **encore allumée** et qu'une **deuxième session utilisateur** semblait active sans que l'occupant du bureau ne soit en mesure de l'expliquer.

Une acquisition mémoire a été réalisée immédiatement à l'aide de *DumpIt*. Le disque dur est en cours d'analyse par une autre équipe ; il vous revient de faire parler la mémoire.

3.2 Travail demandé

Vous rendrez un rapport structuré apportant des réponses argumentées aux questions suivantes.

Q1. Vérifiez l'intégrité du dump remis. Précisez le système d'exploitation, son architecture, la date et l'heure de l'acquisition.

Q2. La machine héberge plus d'un compte utilisateur local. Listez ces comptes. À quel compte la session interactive principale appartient-elle au moment du dump ?

Q3. Plusieurs sessions interactives sont ouvertes simultanément. Pour chacune, indiquez l'utilisateur connecté et l'application au premier plan que vous parvenez à reconstituer.

Q4. L'utilisateur impliqué a manipulé un terminal. Restituez la ligne de commande exacte qu'il a saisie, ainsi que tout fichier qu'il a manipulé via cette ligne de commande.

Q5. Un fichier compressé protégé est présent sur l'une des sessions. Localisez-le, extrayez-le depuis la mémoire, et indiquez son nom et sa taille.

Q6. Le secret nécessaire à l'ouverture du fichier compressé n'est pas écrit en clair. Décrivez la démarche qui vous a permis de le récupérer.

Q7. Le contenu du fichier compressé renferme la **première moitié** d'un flag du type `flag{...}`. Indiquez cette moitié.

Q8. La **seconde moitié** du flag se trouve ailleurs dans la mémoire, dans un artefact graphique manipulé par l'utilisateur. Localisez-la et indiquez-la.

Q9. Reconstituez le flag complet et présentez la chronologie synthétique des actions de l'utilisateur, telles que vous pouvez les défendre devant un magistrat.

4. Exercice 2 — « Profil d'un poste »

Dump associé : `exercice2.raw`

4.1 Contexte

Un poste de travail attribué à un utilisateur identifié uniquement sous le pseudonyme `hello` a été saisi dans le cadre d'une enquête préliminaire. Avant toute mise en accusation, le procureur souhaite disposer d'un **portrait technique exhaustif** de ce poste : son système d'exploitation, son utilisateur, ses applications, son activité réseau récente, et son état précis au moment de l'acquisition. Aucune charge n'est encore retenue ; le travail est **à charge et à décharge**.

Une acquisition mémoire a été réalisée avec *DumpIt* alors que plusieurs fenêtres étaient ouvertes sur le bureau de l'utilisateur.

4.2 Travail demandé

Q1. Vérifiez l'intégrité du dump et identifiez le profil système (famille, version, architecture, date d'acquisition, fuseau).

Q2. Reconstruisez l'arborescence des processus. Précisez la relation parent/enfant, le compte propriétaire de chaque processus utilisateur, et tout processus susceptible de révéler une activité non bureautique en cours.

Q3. Énumérez les comptes locaux présents dans la base SAM en mémoire, avec leur RID et leur empreinte NTLM. Pour le compte principal, **proposez une démarche** de récupération du mot de passe en clair (sans la mettre en œuvre faute d'environnement contrôlé). Quels outils, quels dictionnaires recommandez-vous, et quelle est l'estimation théorique du temps de cassage en regard de la robustesse observée ?

Q4. Reconstituez **l'historique de navigation web** présent en mémoire. Quelles URL témoignent d'un centre d'intérêt particulier de l'utilisateur ? Indiquez la commande exacte utilisée et au moins trois URL pertinentes.

Q5. Cartographiez les **logiciels installés** détectables en mémoire (binaires, librairies, fichiers de configuration). En particulier, l'utilisateur a-t-il un environnement de développement configuré ? Si oui, lequel et avec quels modules visibles ?

Q6. L'utilisateur avait deux instances d'un éditeur de texte ouvertes au moment du dump. Pour chacune, indiquez le PID, la durée de vie en mémoire, et **toute donnée que vous parvenez à extraire** du processus correspondant (chemin de travail, variables d'environnement, fragments d'écriture en cours dans le tampon d'édition). Précisez clairement la **limite méthodologique** : un texte tapé dans Notepad et jamais sauvegardé n'est pas garanti d'être récupérable et son extraction nécessite un travail de carving que vous décrirez.

Q7. L'analyse réseau (ports en écoute + connexions résiduelles) révèle l'adresse IP de la machine et son rôle apparent dans le réseau. Documentez ces éléments.

Q8. L'utilisateur a-t-il interagi récemment avec un **installateur logiciel** (`msiexec.exe`, etc.) ? Si oui, restituez les arguments de ligne de commande et l'état du processus. Que pouvez-vous en déduire ?

Q9. Établissez la **chronologie horodatée** des dix dernières actions notables de l'utilisateur, sur la base des artefacts exclusivement mémoire. Cette chronologie doit pouvoir être défendue lors d'une expertise contradictoire.

Q10. Rédigez **deux règles YARA** : la première qui qualifie les indicateurs neutres (présence de Python, navigation CTF) ; la seconde qui chercherait à détecter une exfiltration en cours (chaîne base64 longue dans la heap utilisateur, par exemple). Justifiez les choix de chaînes et de conditions.

5. Exercice 3 — « Obsession »

Dump associé : `exercice3.raw`

5.1 Contexte

Le poste appartient à un utilisateur principal nommé `eminem`. Un deuxième compte, `SlimShady`, a été créé sur la même machine sans que la hiérarchie en ait été informée. Le SOC a remonté un signal faible suggérant que l'utilisateur principal aurait *« doublé son identité »* afin de compartimenter une activité sensible.

À l'arrivée de l'équipe d'intervention, deux sessions étaient ouvertes en parallèle, dont l'une affichait au premier plan une application de prise de notes natives de Windows.

5.2 Travail demandé

Q1. Authentifiez la preuve et identifiez le profil système.

Q2. Listez l'ensemble des comptes locaux. Pour chaque compte, indiquez son SID, sa date de dernier login en mémoire si vous parvenez à la reconstituer, et l'éventuelle relation hiérarchique entre les comptes.

Q3. L'application de prise de notes affichait une note à l'écran. Reconstituez l'intégralité du contenu de cette note. Quelle information y est-elle inscrite ?

Q4. L'information reconstituée renvoie à un fichier présent en mémoire. Localisez ce fichier et extrayez-le.

Q5. Le fichier extrait contient un objet binaire compressé ou chiffré. Décrivez la structure que vous y identifiez et la procédure permettant d'en extraire le contenu utile.

Q6. Le contenu utile est constitué de plusieurs ressources, dont au moins une image. Cette image porte un message dissimulé. Indiquez la technique de dissimulation employée et restituez le message.

Q7. Reconstituez le flag final, ainsi que la chronologie des actions du second compte. Cette chronologie doit pouvoir être opposée à l'utilisateur principal.

6. Exercice 4 — « Vendredi noir »

Dump associé : `exercice4.raw`

6.1 Contexte

Vendredi 12 mai 2017, en début d'après-midi, le service informatique d'une entreprise est submergé d'appels d'utilisateurs constatant qu'ils ne peuvent plus ouvrir leurs documents et qu'un message inhabituel s'affiche sur leur écran. La direction réquisitionne en urgence l'équipe forensic pour comprendre ce qui se passe sur l'un des postes touchés.

Le dump mémoire qui vous est remis a été acquis sur l'un de ces postes tandis que le phénomène se déroulait. Aucune information additionnelle ne vous est communiquée : il vous revient de **caractériser l'incident** et d'en déduire les actions de remédiation à entreprendre.

6.2 Travail demandé

Q1. Authentifiez la preuve. Identifiez le profil système et expliquez en quoi le choix de l'OS est cohérent avec le contexte historique de l'attaque.

Q2. Listez l'ensemble des processus actifs au moment du dump. Identifiez ceux qui ne correspondent pas à un poste de bureautique légitime, et expliquez pour chacun les éléments qui motivent votre suspicion.

Q3. Restituez la ligne de commande complète associée à chaque processus suspect. Notez en particulier le **chemin disque** depuis lequel le binaire a été lancé. Que pouvez-vous déduire de ce chemin ?

Q4. Examinez les ruches de registre encore présentes en mémoire. Identifiez tout mécanisme de **persistance** mis en place par le code malveillant (clé `Run`, service, tâche planifiée).

Q5. À partir des artefacts disponibles, déterminez le **nom public** de la campagne d'attaque dont la machine est victime. Citez au moins **trois indices indépendants** qui vous permettent d'aboutir à cette conclusion.

Q6. Caractérisez le mode de propagation de cette attaque. Indiquez la vulnérabilité qu'elle exploite ainsi que le **kill switch** historique qui a permis d'enrayer sa progression mondiale.

Q7. Le code malveillant chiffre certains types de fichiers et en laisse d'autres intacts. Identifiez, à partir de la mémoire, deux extensions visées par le code et expliquez la stratégie sous-jacente.

Q8. Rédigez la note d'incident à destination du RSSI : actions immédiates de confinement, actions de remédiation, leçons à tirer pour le parc. Une demi-page suffit.

7. Exercice 5 — « Anatomie d'un ver SMB » *(approfondissement)*

Dump associé : `exercice4.raw` *(le même que l'exercice 4 — angle d'analyse différent)*

> **Niveau attendu** : avancé. Cet exercice s'adresse aux étudiants ayant déjà rendu l'exercice 4 ; il en reprend l'image mais l'aborde sous un angle **réseau / protocolaire**. Le rendu doit faire l'objet d'un compte-rendu **distinct**.

7.1 Contexte

Vous avez livré dans l'exercice 4 un rapport d'incident rapide. Le procureur, satisfait de votre identification, vous demande maintenant une **note technique approfondie** opposable lors d'une expertise contradictoire. L'autre partie, qui pilote des serveurs restés en SMBv1, prétend que la propagation latérale a démarré chez **elle** et que la machine que vous avez analysée n'a fait que recevoir l'attaque. Votre note devra donc démontrer (ou infirmer) :

1. La capacité de la machine à émettre du trafic SMB sortant ;
2. Les traces résiduelles d'une exploitation **EternalBlue** sur le poste analysé (en émission ou en réception) ;
3. La présence éventuelle d'un implant **DoublePulsar** ;
4. Les indicateurs réseau précis qu'un SOC pourrait détecter.

7.2 Travail demandé

Q1. Cartographiez **tous** les ports en écoute (TCP et UDP) au moment du dump. Pour chacun, indiquez le PID propriétaire, le processus, l'adresse IP locale, et précisez si l'écoute est **légitime** ou **suspecte** dans le cadre d'un poste utilisateur.

Q2. Examinez l'état de la pile SMB (`srv.sys` , `mrxsmb*.sys` , `netbt.sys`). Identifiez le ou les drivers en mémoire. Pour chacun, indiquez sa version connue et les vulnérabilités historiques associées.

Q3. Recherchez en mémoire la trace de l'**implant DoublePulsar**. On rappelle que DoublePulsar est un noyau-mode backdoor déposé en ring 0 par EternalBlue, qui répond à des requêtes SMB Trans2 SESSION_SETUP avec un *MultiplexID* particulier. Décrivez la méthode employée (scan de pages noyau, recherche d'opcodes `0x51` / `0x52` dans `srv.sys` , signatures publiques) et le verdict.

Q4. Identifiez la **trame SMB malveillante** caractéristique d'EternalBlue (paquet *Tree Connect AndX* immédiatement suivi d'une suite de *NT Trans / Trans2* fragmentés sur la même session). À partir des buffers réseau encore présents en mémoire (NDIS, `tcpip.sys`), reconstituez un fragment de cette trame.

Q5. Analysez le **comportement de propagation** : la machine a-t-elle initié un balayage de réseau sur le port 445 ? Recherchez les sessions outgoing dans la mémoire de `tasksche.exe` et de la pile réseau, restituez la plage IP scannée si elle existe.

Q6. Le code malveillant tente une résolution DNS du *kill switch*. À partir de la mémoire du résolveur (`dnscache` / `netbt.sys`), retrouvez la trace de cette requête. Précisez : - la chaîne du domaine ; - le processus qui l'a émise ; - le code de retour DNS observé (résolu / non résolu).

Q7. Rédigez **deux règles YARA distinctes** : - une règle « *EternalBlue exploitation residue* » pour pages noyau de `srv.sys` ; - une règle « *DoublePulsar implant* » pour pages exécutables injectées en ring 0.

Pour chaque règle, justifiez le choix des chaînes et des conditions.

Q8. Établissez une liste finale d'**indicateurs de compromission réseau** (IoC réseau) exploitables par un SOC : - ports à filtrer ; - signatures Snort/Suricata existantes (citer la SID si vous la connaissez) ; - domaines / IPs à blackhole ; - processus à terminer.

Q9. Rédigez la conclusion technique opposable. La direction de l'autre partie peut-elle, oui ou non, soutenir que la machine analysée n'a fait que recevoir l'attaque ? Justifiez **strictement** sur la base des artefacts mémoire identifiés.

8. Annexes

8.1 Calculer un SHA-256

```
# macOS / Linux
sha256sum exercice1.raw
shasum -a 256 exercice1.raw
```

8.2 Journaliser une session shell

```
script -q rapport.log
# … vos commandes …
exit
```

8.3 Outils complémentaires autorisés

`strings`, `xxd`, `file`, `binwalk`, `foremost`, `7z`, `unrar`, `john`, `hashcat`, `openssl`, `base64`, `zsteg` (le cas échéant), `steghide`, `exiftool`.

Les recherches en ligne **du nom du fichier de dump** sont considérées comme un manquement à l'intégrité académique.